Daily Quant Digest: 2026-10-18

Today's topics

Track	Topic	Difficulty
Probability & Statistics	Modes of convergence, laws of large numbers, the CLT and when it fails; the St. Petersburg game played a million times	Medium-Hard
Derivatives	Black-Scholes from first principles: replication, the PDE, risk-neutral pricing, and the Garman-Kohlhagen FX version priced on USDJPY	Medium
Algorithms & DS	Heaps: sliding window median with two heaps and lazy deletion	Medium-Hard

1. Probability & Statistics: Convergence, the LLN, and the CLT (and When It Fails)

Interviewers love questions whose "obvious" answer comes from the law of large numbers or the central limit theorem, and then quietly break one of the hypotheses: infinite mean, infinite variance, dependence, or a sample too small for the skew. To spot the trap you need to know exactly what each theorem says and which mode of convergence it delivers.

1.1 Four modes of convergence

Let $X, X_1, X_2, \ldots$ be random variables on one probability space.

Mode	Definition	Notation
Almost surely	$P(\lim_n X_n = X) = 1$	$X_n \xrightarrow{a.s.} X$
In probability	$\forall \varepsilon > 0:\ P(\lvert X_n - X\rvert > \varepsilon) \to 0$	$X_n \xrightarrow{P} X$
In L^p ($p \geq 1$)	$\mathbb{E}\lvert X_n - X\rvert^p \to 0$	$X_n \xrightarrow{L^p} X$
In distribution	$F_{X_n}(x) \to F_X(x)$ at every continuity point of F_X	$X_n \Rightarrow X$

Intuition. Almost sure convergence is about **individual sample paths**: pick an outcome ω, look at the sequence of numbers $X_n(\omega)$, and ask whether it converges. Convergence in probability only asks that, at each large fixed n, a big deviation is unlikely; the rare bad events may keep happening forever, just at different n. Convergence in distribution does not even need the variables on the same space: it is a statement about laws only.

1.2 The implication map and the counterexamples

$$\text{a.s.} \implies \text{in prob} \implies \text{in distribution}, \qquad L^p \implies \text{in prob}.$$

Proofs.

- $L^p \Rightarrow P$: Markov's inequality, $P(|X_n - X| > \varepsilon) \le \mathbb{E}|X_n - X|^p / \varepsilon^p \to 0$.
- *a.s.* $\Rightarrow P$: the indicator $\mathbf{1}\{|X_n - X| > \varepsilon\}$ tends to 0 a.s. and is bounded by 1, so its expectation tends to 0 by dominated convergence.
- $P \Rightarrow$ *distribution*: for continuity point x, $P(X_n \le x) \le P(X \le x + \varepsilon) + P(|X_n - X| > \varepsilon)$, and symmetrically from below; let $n \to \infty$ then $\varepsilon \to 0$.

None of the arrows reverse. Know one counterexample for each; interviewers ask for them.

1. *In probability but not a.s.* Let X_n be independent with $P(X_n = 1) = 1/n$, else 0. Then $P(X_n \neq 0) = 1/n \to 0$, so $X_n \xrightarrow{P} 0$. But $\sum 1/n = \infty$ and the events are independent, so by the **second Borel-Cantelli lemma** $X_n = 1$ infinitely often, a.s. The path never settles.
2. *a.s. but not in L^1.* With $U \sim \text{Unif}(0,1)$, let $X_n = n\,\mathbf{1}\{U < 1/n\}$. For each fixed $U > 0$, $X_n = 0$ once $n > 1/U$, so $X_n \to 0$ a.s. But $\mathbb{E}X_n = 1$ for all n. The mass escapes to infinity on a shrinking set. The missing condition is **uniform integrability**: $\sup_n \mathbb{E}[|X_n| \mathbf{1}\{|X_n| > M\}] \to 0$ as $M \to \infty$. Vitali's theorem: $X_n \xrightarrow{P} X$ plus UI $\iff$ $X_n \xrightarrow{L^1} X$.
3. *In distribution but not in probability.* $X \sim N(0,1)$ and $X_n = -X$ for all n. Same law every time, but $|X_n - X| = 2|X|$ never shrinks.

Partial converses worth knowing.

- In probability implies a.s. along a **subsequence** (choose n_k with $P(|X_{n_k} - X| > 2^{-k}) < 2^{-k}$ and apply the first Borel-Cantelli lemma).
- In distribution to a **constant** c implies in probability to c.
- **Borel-Cantelli.** (I) If $\sum P(A_n) < \infty$ then $P(A_n \text{ i.o.}) = 0$, no independence needed. (II) If the A_n are independent and $\sum P(A_n) = \infty$, then $P(A_n \text{ i.o.}) = 1$. Almost every a.s. statement in this note is proved with (I).

1.3 Laws of large numbers

Let $X_1, X_2, \ldots$ be i.i.d., $S_n = X_1 + \cdots + X_n$, $\bar{X}_n = S_n/n$.

Weak LLN (finite variance), proof in one line. $\mathrm{Var}(\bar X_n) = \sigma^2/n$, so Chebyshev gives $P(|\bar X_n - \mu| > \varepsilon) \le \sigma^2/(n\varepsilon^2) \to 0$.

Khinchin's WLLN needs only $\mathbb{E}|X| < \infty$. Proof sketch via characteristic functions: $\varphi_{\bar X_n}(t) = \varphi(t/n)^n = (1 + i\mu t/n + o(1/n))^n \to e^{i\mu t}$, the characteristic function of the constant μ; convergence in distribution to a constant is convergence in probability.

Kolmogorov's strong LLN. $\bar X_n \to \mu$ a.s. **if and only if** $\mathbb{E}|X| < \infty$ (and then $\mu = \mathbb{E}X$).

Proof sketch under a fourth moment (the version to reproduce at a whiteboard). Centre so $\mu = 0$. Expanding $\mathbb{E}[S_n^4]$, every term containing a lone X_i vanishes, leaving $n\,\mathbb{E}X^4 + 3n(n-1)\sigma^4 \le Cn^2$. Markov: $P(|\bar X_n| > \varepsilon) \le Cn^2/(n^4\varepsilon^4) = C/(n^2\varepsilon^4)$, which is summable. Borel-Cantelli (I): $|\bar X_n| > \varepsilon$ only finitely often, a.s., for each rational ε. $\square$

The converse is the important part for interviews. If $\mathbb{E}|X| = \infty$ then $\sum_n P(|X_n| > Cn) = \infty$ for every C (since $\sum_n P(|X| > Cn) \ge \mathbb{E}|X|/C - 1$), so by Borel-Cantelli (II) $|X_n| > Cn$ infinitely often. A single term of size Cn moves the average by C, so $\bar X_n$ cannot converge to any finite limit.

1.4 The central limit theorem

Theorem (Lindeberg-Levy). If X_i are i.i.d. with mean μ and variance $\sigma^2 \in (0, \infty)$, then

$$\frac{S_n - n\mu}{\sigma\sqrt{n}} \Rightarrow N(0,1).$$

Proof sketch. Standardise so $\mu = 0$, $\sigma = 1$. A finite second moment gives the Taylor expansion $\varphi(s) = 1 - s^2/2 + o(s^2)$. Then

$$\varphi_{S_n/\sqrt{n}}(t) = \varphi(t/\sqrt{n})^n = \left(1 - \frac{t^2}{2n} + o\big(\tfrac{1}{n}\big)\right)^n \to e^{-t^2/2},$$

and Levy's continuity theorem (from the generating functions digest) converts this into convergence in distribution. $\square$

Notice exactly where variance enters: it is the s^2 coefficient. If $\mathbb{E}X^2 = \infty$ the expansion fails and so does the theorem.

Lindeberg-Feller (independent, not identical). With $s_n^2 = \sum \mathrm{Var}(X_i)$, the CLT holds if for every $\varepsilon > 0$

$$\frac{1}{s_n^2}\sum_{i=1}^{n} \mathbb{E}\big[(X_i - \mu_i)^2\,\mathbf{1}\{|X_i - \mu_i| > \varepsilon s_n\}\big] \to 0.$$

In words: **no single term may carry a non-negligible share of the variance.** This is the condition that fails when one jump dominates.

Berry-Esseen (how fast). With $\rho = \mathbb{E}|X-\mu|^3$,

$$\sup_x \left| P\left(\tfrac{S_n - n\mu}{\sigma\sqrt{n}} \le x\right) - \Phi(x) \right| \le \frac{C\rho}{\sigma^3\sqrt{n}}, \qquad C \le 0.4748.$$

The error is $O(1/\sqrt{n})$ and scales with **skewness**. Symmetric, light-tailed summands converge fast; lottery-like summands converge painfully slowly.

Two tools that ride on top of the CLT.

- *Slutsky:* if $X_n \Rightarrow X$ and $Y_n \xrightarrow{P} c$, then $X_n + Y_n \Rightarrow X + c$ and $X_n Y_n \Rightarrow cX$. This is why you may plug in the sample standard deviation and still get a normal t-statistic for large n.
- *Delta method:* if $\sqrt{n}(\bar{X}_n - \mu) \Rightarrow N(0, \sigma^2)$ and $g'(\mu) \neq 0$, then $\sqrt{n}\,(g(\bar{X}_n) - g(\mu)) \Rightarrow N(0, g'(\mu)^2\sigma^2)$. Example: the standard error of an estimated Sharpe ratio, or of log of a sample variance.

1.5 When the CLT fails (or lies)

1. **Infinite variance: stable laws.** If X is standard Cauchy, $\varphi(t) = e^{-|t|}$, so $\varphi_{\bar{X}_n}(t) = e^{-n|t/n|} = e^{-|t|}$: **the average of n Cauchys is again standard Cauchy.** Averaging buys nothing. More generally, for tails $P(|X| > x) \sim cx^{-\alpha}$ with $\alpha < 2$, the generalised CLT says $(S_n - b_n)/n^{1/\alpha}$ converges to an α-stable law, not a normal, and the scaling is $n^{1/\alpha}$ rather than $\sqrt{n}$.
2. **Infinite mean.** For $\alpha \le 1$ even the LLN fails (Section 1.3 converse). The interview question below lives here.
3. **Dependence.** For a stationary series with autocorrelations ρ_k, the variance of S_n is roughly $n\sigma^2(1 + 2\sum_{k\ge1}\rho_k)$, not $n\sigma^2$. Positively autocorrelated P&L (momentum, stale marks, overlapping returns) makes naive t-stats too large; this is why Newey-West standard errors exist. With long memory the sum may not even be $O(\sqrt{n})$.
4. **One term dominates (Lindeberg fails).** Sum of independent terms where the k-th has variance 2^k: the last term carries half the total variance, and the limit is whatever shape that last term has.
5. **Small n with heavy skew.** Take $S \sim \mathrm{Bin}(100, 0.01)$ (100 independent 1%-default bonds). Exact $P(S \ge 3) = 0.0794$; the normal approximation with continuity correction gives 0.0658, the Poisson(1) gives 0.0803. Exact $P(S = 0) = 0.366$ versus normal 0.308. The Berry-Esseen bound here is 0.47, which is useless. With $np = 1$ you are in the Poisson regime, not the Gaussian one.
6. **Tails of sums.** The CLT describes deviations of order $\sqrt{n}$. For deviations of order n (the probability your book loses 5 standard deviations) the Gaussian tail is wrong even for moderate n; you need large-deviation estimates (the Chernoff bound from the inequalities digest).

1.6 Common pitfalls

- "The sample mean converges, so its expectation converges." No: a.s. convergence does not give L^1 convergence without uniform integrability (counterexample 2).
- "With enough data the CLT always kicks in." Only with finite variance, and the required n grows with skewness.
- "$\mathbb{E}[X] = \infty$ means you should pay any price." The expectation is a statement about the LLN limit, which does not exist here. What you will actually receive over n plays is a different, finite, quantity.
- Mixing up modes: the SLLN is an a.s. statement about one infinite path; Feller-type laws (below) are only in probability.

1.7 Interview question

> **The St. Petersburg game, played a million times.** A fair coin is tossed until the first head. If the first head appears on toss k, you receive $\$2^k$.
>
> a. What is the expected payout of one game?
>
> b. You are offered a bundle of $n = 2^{20} \approx 10^6$ independent games and are paid the total. Roughly what will your **average payout per game** be? How much per game would you pay for the bundle?
>
> c. Why doesn't "the average converges to the mean" answer (b)?

1.8 Solution

Step 1: the distribution. Let X be one payout. $P(X = 2^k) = 2^{-k}$ for $k = 1, 2, \dots$ (first $k-1$ tosses tails, then a head). So $X \in \{2, 4, 8, \dots\}$.

Step 2: (a) the mean.

$$\mathbb{E}X = \sum_{k \geq 1} 2^k \cdot 2^{-k} = \sum_{k \geq 1} 1 = \infty.$$

Each level contributes exactly \$1 to the expectation, and there are infinitely many levels.

Step 3: (c) why the LLN fails. Kolmogorov's SLLN requires $\mathbb{E}|X| < \infty$. By the converse in 1.3, the running average $\bar{X}_n$ is hit infinitely often by single payouts larger than Cn for any C, so $\bar{X}_n \to \infty$ a.s. (it is nonnegative and has unbounded jumps), but it does not converge to any number and the expectation tells you nothing about its size at a given n.

Step 4: the tail. We need tail probabilities. If j is the smallest integer with $2^j > b$, then

$$P(X > b) = \sum_{k \geq j} 2^{-k} = 2^{1-j} < \frac{2}{b}.$$

So $P(X > b) \asymp 1/b$: a tail index $\alpha = 1$, exactly the borderline where the mean diverges (logarithmically).

Step 5: the interview trick, "count by levels". In n games, about $n/2^k$ games end at level k and each pays 2^k, so **each level contributes about n dollars** to the total. This bookkeeping is reliable as long as the expected count $n/2^k$ is large, i.e. for k up to about $\log_2 n$. Beyond that level, outcomes are lottery tickets: maybe one game reaches level $\log_2 n + 1$ and pays about $2n$, maybe none does. So

$$S_n \approx n \log_2 n + (\text{a random term of order } n), \qquad \bar{X}_n \approx \log_2 n + O(1).$$

For $n = 2^{20}$: about **\$20 per game**, plus a random, positively skewed correction of a few dollars.

Step 6: making it rigorous (Feller's weak law). Claim: $\dfrac{S_n}{n \log_2 n} \xrightarrow{P} 1$.

Truncate at $b_n = n \log_2 n$: let $Y_i = X_i \, \mathbf{1}\{X_i \le b_n\}$ and $T_n = \sum Y_i$.

- *Truncation is harmless.* $P(S_n \neq T_n) \le n \, P(X > b_n) < \dfrac{2n}{n \log_2 n} = \dfrac{2}{\log_2 n} \to 0.$
- *Mean of the truncated variable.* Each level k with $2^k \le b_n$ contributes 1: $\mathbb{E}Y = \lfloor \log_2 b_n \rfloor = \log_2 n + \log_2 \log_2 n + O(1)$, so $\mathbb{E}T_n / (n \log_2 n) \to 1$.
- *Variance of the truncated variable.* $\mathbb{E}Y^2 = \sum_{2^k \le b_n} 2^{2k} 2^{-k} = \sum_{2^k \le b_n} 2^k < 2b_n$, so $\mathrm{Var}(T_n) < 2nb_n = 2n^2 \log_2 n$.
- *Chebyshev.* $P\big(|T_n - \mathbb{E}T_n| > \varepsilon \, n \log_2 n\big) \le \dfrac{2n^2 \log_2 n}{\varepsilon^2 n^2 (\log_2 n)^2} = \dfrac{2}{\varepsilon^2 \log_2 n} \to 0.$

Combine the three bullets: $S_n / (n \log_2 n) \to 1$ in probability. □

Note what kind of statement this is: convergence **in probability**, with an unusual normalisation $n \log n$ instead of n, and very slow (the error terms are $1/\log n$). It is not an a.s. statement: along a single infinite path, $S_n/(n \log_2 n)$ has $\limsup = \infty$ almost surely (Chow-Robbins). This is a beautiful example where weak and strong laws genuinely differ.

Step 7: (b) the number. A simulation with 200,000 independent bundles (sampling the level counts as sequential binomials) gives, for the average payout per game:

n	$\log_2 n$	**10th pct**	**25th pct**	**median**	**75th pct**	**90th pct**	$P(\bar{X}_n < \log_2 n)$
2^{10}	10	9.11	10.36	12.60	17.17	27.31	0.20
2^{20}	20	19.06	20.34	22.62	27.19	37.31	0.21
2^{30}	30	29.06	30.33	32.58	37.12	47.23	0.21

So $\bar{X}_n - \log_2 n$ settles into a fixed, right-skewed distribution (median about 2.6) along $n = 2^m$. This matches the level-counting picture: the "bulk" contributes $\log_2 n$, the lottery levels contribute an $O(1)$ random amount per game.

Answer. For a million games you should expect to collect around **\$20 to \$23 per game** (median about \$22.6); a risk-neutral-ish bidder for the bundle might pay roughly \$21 to \$22 per game, a risk-averse one less. The expected value is infinite, but it is carried by astronomically unlikely paths.

Sanity checks.

- Doubling n should add about \$1 per game (one more level is "populated"). The table: each factor 2^{10} adds 10. ✓
- The answer must exceed \$2 (minimum payout) and grow without bound in n, but slowly. ✓
- Variance check: the truncation step says the relative noise is at most $\sqrt{2/\log_2 n} \approx 0.32$ at $n = 2^{20}$; the simulated interquartile range of ± 3 around 22 is within that. ✓

Follow-ups to expect. "What if payouts were capped at \$1bn?" (Then $\mathbb{E}X = \lfloor \log_2 10^9 \rfloor + P(X > 10^9) \cdot 10^9 \approx 29 + 1.86 \approx \$31$: the paradox is about unbounded tails.) "What is the utility-based answer?" (With log utility, $\mathbb{E} \log X = \sum k \log 2 \cdot 2^{-k} = 2 \log 2$, a certainty equivalent of \$4 per single game, Bernoulli's original resolution.)

---

2. Derivatives: Black-Scholes from First Principles (and Garman-Kohlhagen for FX)

Everything in options trading is quoted through Black-Scholes, even though nobody believes its assumptions. You need to know **why** the formula is what it is: the price comes from **replication**, not from forecasting, and that is why the stock's expected return does not appear.

2.1 Why expected payoff is the wrong answer

A natural first guess: option value \$= \$ discounted real-world expected payoff. The problem is that this depends on the drift μ, which nobody agrees on, and on the discount rate, which should depend on the option's risk. Black, Scholes and Merton's insight: if the option's payoff can be **manufactured** from the stock and cash, its price must equal the cost of manufacturing it, or there is an arbitrage. Then nobody's opinion on μ matters.

2.2 One-period replication (the whole idea in five lines)

Stock at 100; in one period it goes to 120 or 80; interest rate 0; call struck at 100 pays 20 or 0.

Hold Δ shares and B in cash so that the portfolio matches the call in both states:

$$120\Delta + B = 20, \qquad 80\Delta + B = 0 \;\Rightarrow\; \Delta = \frac{20 - 0}{120 - 80} = 0.5, \quad B = -40.$$

Cost today: $0.5 \times 100 - 40 = 10$. **The call is worth 10**, whatever the real probability of going up. If it trades at 12, sell it, buy the replicating portfolio for 10, pocket 2, and every state nets to zero.

Write the price as an expectation: $10 = q \cdot 20 + (1 - q) \cdot 0$ with $q = 0.5$. In general $q = \dfrac{(1 + r) - d}{u - d}$, the unique probability under which the stock earns the riskless rate. This is the **risk-neutral measure** $\mathbb{Q}$: not a belief, just the bookkeeping device that makes prices equal discounted expectations.

2.3 Continuous time: the hedging argument and the PDE

Assumptions. The stock follows geometric Brownian motion $dS = \mu S\,dt + \sigma S\,dW$; constant r, σ, dividend yield q; frictionless continuous trading; no arbitrage.

Ito's lemma (the one stochastic calculus fact needed: $(dW)^2 = dt$, so second-order terms in dS survive): for $V(S, t)$,

$$dV = \left(V_t + \mu S V_S + \tfrac{1}{2}\sigma^2 S^2 V_{SS}\right)dt + \sigma S V_S\,dW.$$

Hedge. Hold the option and short Δ shares: $\Pi = V - \Delta S$. Over dt the short position also pays the dividend $q\Delta S\,dt$:

$$d\Pi = dV - \Delta\,dS - q\Delta S\,dt.$$

Choose $\Delta = V_S$. The dW terms cancel, **and so do the μ terms**:

$$d\Pi = \left(V_t + \tfrac{1}{2}\sigma^2 S^2 V_{SS} - qSV_S\right)dt.$$

A riskless portfolio must earn the riskless rate, $d\Pi = r\Pi\,dt = r(V - SV_S)\,dt$. Equating gives the **Black-Scholes PDE**:

$$V_t + (r - q)SV_S + \tfrac{1}{2}\sigma^2 S^2 V_{SS} - rV = 0, \qquad V(S, T) = \text{payoff}(S).$$

Read it as the trader's identity $\Theta + \tfrac{1}{2}\sigma^2 S^2 \Gamma + (\text{carry terms}) = rV$: a delta-hedged long option pays theta to earn gamma. That trade-off is the subject of the upcoming Greeks and gamma scalping digests.

2.4 Risk-neutral pricing and the formula

By the Feynman-Kac theorem, the PDE solution is

$$V(S,0) = e^{-rT}\,\mathbb{E}^{\mathbb{Q}}[\text{payoff}(S_T)], \qquad dS = (r-q)S\,dt + \sigma S\,dW^{\mathbb{Q}}.$$

Under $\mathbb{Q}$, $S_T = Fe^{-\frac{1}{2}\sigma^2 T + \sigma\sqrt{T}Z}$ with $F = Se^{(r-q)T}$ the forward and $Z \sim N(0,1)$.

The call. $\mathbb{E}^{\mathbb{Q}}[(S_T - K)^+] = \mathbb{E}[S_T \mathbf{1}\{S_T > K\}] - K\,\mathbb{Q}(S_T > K)$.

- $\mathbb{Q}(S_T > K) = P\big(Z > \frac{\ln(K/F) + \frac{1}{2}\sigma^2 T}{\sigma\sqrt{T}}\big) = N(d_2)$.
- For the first term, write $\mathbb{E}[S_T \mathbf{1}\{\cdot\}] = F\,\mathbb{E}[e^{\sigma\sqrt{T}Z - \frac{1}{2}\sigma^2 T}\mathbf{1}\{\cdot\}]$. The factor $e^{\sigma\sqrt{T}Z - \frac{1}{2}\sigma^2 T}$ is a density that shifts Z to $Z + \sigma\sqrt{T}$ (complete the square), so this term is $F\,N(d_2 + \sigma\sqrt{T}) = F\,N(d_1)$.

Hence

$$\boxed{C = e^{-rT}\big[F\,N(d_1) - K\,N(d_2)\big], \quad P = e^{-rT}\big[K\,N(-d_2) - F\,N(-d_1)\big], \quad d_{1,2} = \frac{\ln(F/K)}{\sigma\sqrt{}}}$$

Interpretations. $N(d_2)$ is the risk-neutral probability of exercise, so $e^{-rT}N(d_2)$ is the price of a digital call. $e^{-qT}N(d_1)$ is the spot delta. $C - P = e^{-rT}(F - K)$ is put-call parity (yesterday's topic) falling out automatically.

2.5 FX: Garman-Kohlhagen

Holding foreign currency earns the foreign interest rate, exactly like a dividend yield. So for a pair quoted as FOR/DOM (units of domestic per one unit of foreign), set $r = r_d$ and $q = r_f$:

$$F = S\,e^{(r_d - r_f)T}, \qquad C = e^{-r_d T}\big[FN(d_1) - KN(d_2)\big].$$

For **USDJPY**, USD is the foreign (base) currency and JPY the domestic (terms) currency. The formula gives a price in **JPY per 1 USD of notional** ("JPY pips"); dividing by spot converts to **USD % of notional**, the usual quoting for USDJPY options. Which currency the premium is paid in also changes the delta you should hedge with (premium-adjusted delta), a topic for the FX conventions day.

2.6 Worked scenario: pricing and hedging a 3M USDJPY call

You buy a **3-month USD call / JPY put, USD 10M notional, strike 150.00**, from a dealer. Illustrative market:

Input	Value
Spot S	150.00
T	0.25 years
Vol σ	10.0%
JPY rate r_d	0.75% (cont. comp.)
USD rate r_f	3.75% (cont. comp.)

Step 1: forward. $F = 150 \times e^{(0.0075-0.0375)(0.25)} = 150\,e^{-0.0075} = 148.8792$. Forward points -1.1208 (USD at a forward discount because USD rates are higher). The strike of 150 is **at-the-money spot but out-of-the-money forward** for a USD call.

Step 2: d_1, d_2**.** $\sigma\sqrt{T} = 0.10 \times 0.5 = 0.05$; $\ln(F/K) = -0.0075$.

$$d_1 = \frac{-0.0075 + 0.00125}{0.05} = -0.125, \qquad d_2 = -0.175.$$

$N(d_1) = 0.45026$, $N(d_2) = 0.43054$.

Step 3: price.

$$C = e^{-0.001875}\left[148.8792 \times 0.45026 - 150 \times 0.43054\right] = 2.4490 \text{ JPY per USD}.$$

Premium on USD 10M: **JPY 24.49M**, or USD 163,270 at spot, i.e. **1.633% of USD notional**.

Parity check: the 150 put is worth 3.5677, and $C - P = -1.1187 = e^{-r_d T}(F - K)$. ✓

Step 4: delta hedge. Spot delta $= e^{-r_f T} N(d_1) = 0.4461$. Long the USD call means long USD 4.46M of delta, so to be flat you **sell USD 4.46M / buy JPY spot** (forward delta $N(d_1) = 0.4503$ if you hedge with the 3M outright).

Step 5: the other Greeks.

- Gamma $= e^{-r_f T} n(d_1)/(S\sigma\sqrt{T}) = 0.0523$ per JPY: if USDJPY rallies one big figure to 151.00, your delta grows by about $0.0523 \times 10\text{M} \approx$ USD 0.52M, which you must sell to re-hedge (selling high: that is the gamma P&L).
- Vega $= S e^{-r_f T} n(d_1)\sqrt{T} = 29.41$ JPY per USD per unit vol, so **JPY 2.94M (about USD 19.6k) per vol point** on the 10M.

Step 6: quick checks. The ATM-forward rule of thumb $C \approx 0.4\,\sigma\sqrt{T}\,F e^{-r_d T}$ gives $0.4 \times 0.05 \times 148.88 \times 0.998 = 2.972$; exact ATMF (strike $= F$) is 2.964. Our strike is 1.12 above the forward with delta about 0.45, so a price below 2.96 (we got 2.449, roughly ATMF minus delta times the distance, $2.964 - 0.45 \times 1.12 \approx 2.46$) is consistent. ✓

2.7 How good is the replication? Discrete hedging

Black-Scholes assumes continuous rebalancing. Simulating 20,000 GBM paths at exactly 10% vol and delta-hedging the option above at discrete intervals (premium invested at r_d, hedge carried at r_f), the terminal hedging error per USD of notional is:

Rebalances over 3M	Mean error (JPY)	Std dev (JPY)	Std / premium
13 (weekly)	0.004	0.696	28%
63 (daily, business)	0.005	0.321	13%
252 (4x daily)	0.000	0.164	6.7%

The mean is zero (the formula is right on average) but the noise shrinks only like $1/\sqrt{N}$: four times as many hedges halve the error. On USD 10M, daily hedging still leaves a one-standard-deviation outcome of about JPY 3.2M, which is more than a vol point of vega.

2.8 What can go wrong (where the model breaks)

- **Volatility is not constant.** Realised vol differs from implied; the market charges different vols for different strikes (the smile). In practice Black-Scholes is a **quoting map** from price to implied vol, not a belief.
- **Jumps.** USDJPY gaps on intervention headlines or carry unwinds (as in August 2024). A gap cannot be delta-hedged; the hedge argument in 2.3 assumes continuous paths.
- **Transaction costs and discreteness** (2.7): every rebalance crosses a spread; hedging more often reduces variance but raises cost.
- **Rates are not constant**, which matters more for long-dated FX options where the forward itself is volatile.
- **Wrong delta convention.** In FX, hedging with spot delta when the trade is premium-adjusted, or with spot when you meant forward, leaves a residual position.

2.9 Likely interview questions

1. **Why does the stock's expected return not appear in the Black-Scholes price?** Because the option is replicated by a dynamic stock-and-cash portfolio whose cost does not depend on μ; in the hedge, the μ terms cancel when $\Delta = V_S$. μ affects the probability of exercise in the real world, but the hedge's P&L offsets it exactly.
2. **Price a 1-year at-the-money-forward call on a 20%-vol stock at 100, zero rates, in your head.**
   $0.4 \times 0.20 \times 1 \times 100 = 8.0$ (exact: $100(2N(0.1) - 1) = 7.97$). The rule comes from $N(x) \approx \frac{1}{2} + x/\sqrt{2\pi}$ and $1/\sqrt{2\pi} \approx 0.4$.
3. **What are $N(d_1)$ and $N(d_2)$?**
   $N(d_2)$ is the risk-neutral probability the call finishes in the money ($-\partial C/\partial K$ undiscounted); $N(d_1)$ is the forward delta, equivalently the exercise probability under the measure that uses the stock as numeraire.

3. Algorithms & Data Structures: Sliding Window Median

3.1 Problem

Given an integer array `nums` and a window size `k`, return the median of every contiguous window of length `k`, from left to right. For even `k` the median is the mean of the two middle values.

Constraints. $1 \le k \le n \le 10^5$, $-2^{31} \le$ `nums[i]` $\le 2^{31}-1$. Answers within 10^{-5} are accepted.

Examples.

```
nums = [1, 3, -1, -3, 5, 3, 6, 7], k = 3  ->  [1, -1, -1, 3, 5, 6]
nums = [1, 3, -1, -3, 5, 3, 6, 7], k = 4  ->  [0, 1, 1, 4, 5.5]
nums = [2147483647, 2147483647],   k = 2  ->  [2147483647]
```

A rolling median of trade prices is a standard robust filter for bad ticks: unlike a rolling mean, one fat-fingered print cannot move it far. Interviewers at trading firms like this problem because it combines heaps, invariants and a deletion trick.

3.2 From first principles

Brute force. Copy each window and sort it: $O(nk \log k)$, about 10^{10} operations at the limits. Too slow.

Better brute force. Keep the window as a sorted `vector`. Each step, binary-search to remove the outgoing value and insert the incoming one. Searching is $O(\log k)$ but shifting elements is $O(k)$, so $O(nk)$ overall. Still too slow for $k \approx n/2$.

Insight 1: the median only needs a split, not a full sort. Maintain two halves of the window:

- `lo`: the smaller half, as a **max-heap** (its top is the largest small element),
- `hi`: the larger half, as a **min-heap** (its top is the smallest large element),

with the invariants (i) every element of `lo` $\le$ every element of `hi`, and (ii) $|\text{lo}| = |\text{hi}|$ or $|\text{lo}| = |\text{hi}| + 1$. Then the median is `lo.top()` for odd k, or the mean of the two tops for even k. Inserting is $O(\log k)$: push into the correct side, then if the sizes are off by more than allowed, move one top across. This alone solves “median of a data stream”.

The obstacle: deletion. In a sliding window the oldest value must leave, but a binary heap only removes its top. Removing an arbitrary element would need its position.

Insight 2: lazy deletion. We do not have to remove the outgoing element **now**. We only ever look at the two tops. So:

- record the value in a hash map `delayed[x]++` (“one copy of x is dead”),

- adjust the **logical** size of the half it belongs to,
- whenever a dead element reaches the top of a heap, pop it (`prune`).

Which half does x belong to? If $x \le$ `lo.top()` it is (logically) in `lo`, otherwise in `hi`. Duplicates are not a problem: if copies of x sit in both heaps then x equals `lo.top()`, and since the copies are interchangeable it does not matter which physical one dies.

The invariant that makes it work: after every public operation, *both tops are live*. We prune whenever a top might have become dead: after erasing a value equal to a top, and after popping a top during rebalancing.

Complexity. Each element is pushed at most a constant number of times (once on insertion, plus possible moves) and popped at most once per push, each $O(\log n)$ since dead elements may linger in the heaps. Total $O(n \log n)$ **time**, $O(n)$ **space** in the worst case (dead elements buried below the tops). The ordered-multiset alternative in 3.5 achieves $O(n \log k)$ time and $O(k)$ space.

3.3 C++ solution

```
#include <algorithm>
#include <cassert>
#include <cmath>
#include <climits>
#include <functional>
#include <iostream>
#include <queue>
#include <random>
#include <unordered_map>
#include <vector>

class DualHeap {
public:
    explicit DualHeap(int k) : k_(k) {}

    void insert(int x) {
        if (lo_.empty() || x <= lo_.top()) {
            lo_.push(x);
            ++loSize_;
        } else {
            hi_.push(x);
            ++hiSize_;
        }
        rebalance();
    }

    void erase(int x) {
        ++delayed_[x];                 // remember to throw x away later
        if (x <= lo_.top()) {          // x lives (logically) in the lower half
            --loSize_;
            if (x == lo_.top()) prune(lo_);
        } else {                       // x lives in the upper half
            --hiSize_;
            if (x == hi_.top()) prune(hi_);
        }
        rebalance();
    }

    double median() const {
```

```
        if (k_ % 2 == 1) return static_cast<double>(lo_.top());
        // promote to double before adding: INT_MAX + INT_MAX overflows int
        return (static_cast<double>(lo_.top()) + static_cast<double>(hi_.top())) / 2.0;
    }

private:
    // Pop every stale element sitting on top of heap h.
    template <typename Heap>
    void prune(Heap& h) {
        while (!h.empty()) {
            auto it = delayed_.find(h.top());
            if (it == delayed_.end() || it->second == 0) break;
            if (--(it->second) == 0) delayed_.erase(it);
            h.pop();
        }
    }

    // Keep loSize_ == hiSize_ or loSize_ == hiSize_ + 1.
    void rebalance() {
        if (loSize_ > hiSize_ + 1) {
            hi_.push(lo_.top());
            lo_.pop();
            --loSize_;
            ++hiSize_;
            prune(lo_);
        } else if (loSize_ < hiSize_) {
            lo_.push(hi_.top());
            hi_.pop();
            ++loSize_;
            --hiSize_;
            prune(hi_);
        }
    }

    int k_;
    std::priority_queue<int> lo_;                                        // max-heap: smaller half
    std::priority_queue<int, std::vector<int>, std::greater<int>> hi_; // min-heap: larger half
    std::unordered_map<int, int> delayed_;                               // value -> pending
deletions
    int loSize_ = 0, hiSize_ = 0;                                        // logical (valid) sizes
};

std::vector<double> medianSlidingWindow(const std::vector<int>& nums, int k) {
    DualHeap dh(k);
    for (int i = 0; i < k; ++i) dh.insert(nums[i]);
    std::vector<double> ans;
    ans.reserve(nums.size() - k + 1);
    ans.push_back(dh.median());
    for (std::size_t i = k; i < nums.size(); ++i) {
        dh.insert(nums[i]);
        dh.erase(nums[i - k]);
        ans.push_back(dh.median());
    }
    return ans;
}
```

The full file `2026-10-18.cpp` adds a `main` that checks the three examples above, a duplicates case, and 3,000 random arrays (values in a small range so duplicates are frequent) against the sort-every-window brute force. All pass.

Trace of the first example ($k = 3$). Insert 1, 3, -1: `lo = {1, -1}`, `hi = {3}`, median `lo.top() = 1`. Next window: insert -3 into `lo` (sizes 3 and 1, too lopsided), so move 1 across: `lo = {-1, -3}`, `hi = {1, 3}`. Erase 1: since $1 > $ `lo.top()` $= -1$ it belongs to `hi`;

`hiSize` drops to 1, and because 1 is `hi.top()` it is pruned immediately. Now `lo = {-1, -3}`, `hi = {3}`, median -1. ✓

3.4 Walkthrough of the C++

- **`std::priority_queue<int>`** is a binary heap adaptor over a `std::vector<int>`, and by default it is a **max-heap** (it uses `std::less`, and the "largest" element sits on top). Operations: `push` $O(\log n)$, `pop` $O(\log n)$ (removes the top, returns nothing), `top` $O(1)$, `empty`, `size`. There is **no** erase-by-value or iteration in order, which is exactly why we need lazy deletion.
- **`std::priority_queue<int, std::vector<int>, std::greater<int>>`** is the min-heap. The three template arguments are the element type, the underlying container, and the comparator. The comparator semantics feel inverted: `greater` makes the **smallest** element the top. You must spell out the container (`std::vector<int>`) to be allowed to pass the third argument.
- **`explicit DualHeap(int k) : k_(k) {}`**: a constructor with a member initialiser list. `explicit` prevents accidental implicit conversion from `int` to `DualHeap` (e.g. passing `5` where a `DualHeap` is expected).
- **Default member initialisers** `int loSize_ = 0, hiSize_ = 0;` (C++11) mean every constructor starts them at zero without repeating it. The trailing underscore is just a common naming convention for private members.
- **`template <typename Heap> void prune(Heap& h)`**: a member function template, so one body works for both heap types (which are different C++ types because their comparators differ). `Heap&` is a **reference**: we modify the caller's heap, not a copy. Passing by value would compile and silently do nothing useful.
- **`delayed_.find(h.top())`** returns an iterator to the key-value pair or `delayed_.end()`. `it->second` is the count. We use `find` instead of `delayed_[h.top()]` because `operator[]` **inserts** a zero entry when the key is missing, slowly bloating the map. When a count reaches zero we `erase(it)` to keep the map small.
- **`--(it->second) == 0`** decrements the count in place through the iterator and tests the new value in one expression.
- **`double median() const`**: `const` promises the function does not modify the object, so it can be called on a `const DualHeap&`. `top()` on a `priority_queue` has a `const` overload, so this compiles.
- **`static_cast<double>(...)`** before the addition is essential: `INT_MAX + INT_MAX` in `int` is signed overflow, which is **undefined behaviour** in C++, not just a wrong answer. Converting each operand first makes the addition happen in `double` (exact for all 32-bit integers).
- **`const std::vector<int>& nums`**: pass large inputs by const reference to avoid a copy. `ans.reserve(...)` preallocates capacity so `push_back` never reallocates.
- **`std::size_t i = k`** with `nums.size()`: comparing a signed `int` against the unsigned `size()` triggers `-Wall` warnings (and real bugs when the int is negative), so the loop index is unsigned.

- **In `main`**: `assert((r1 == std::vector<double>{...}))` needs the **double parentheses** because `assert` is a macro, and the comma inside the braces would otherwise split it into two macro arguments. `std::vector<int> w(nums.begin() + i, nums.begin() + i + k)` is the iterator-range constructor (copies a half-open range). `std::mt19937 rng(42)` is a seeded Mersenne Twister so the random tests are reproducible, and `for (int& x : v)` is a range-for with a reference so the assignment writes into the vector.

3.5 Variations to be ready for

- **Median of a data stream** (insert only, query any time): the same two heaps without `delayed_`; $O(\log n)$ insert, $O(1)$ query.
- **Ordered multiset with a middle iterator** ($O(n \log k)$, $O(k)$ memory). A `std::multiset` is a balanced BST that supports erase of a specific element, and its iterators stay valid across inserts and erases of other elements:

```
std::vector<double> medianMultiset(const std::vector<int>& nums, int k) {
    std::multiset<int> w(nums.begin(), nums.begin() + k);
    auto mid = std::next(w.begin(), k / 2);          // points at the upper median
    std::vector<double> ans;
    for (std::size_t i = k;; ++i) {
        ans.push_back((static_cast<double>(*mid) + *std::prev(mid, 1 - k % 2)) / 2.0);
        if (i == nums.size()) break;
        w.insert(nums[i]);                           // equal keys go after existing ones
        if (nums[i] < *mid) --mid;
        if (nums[i - k] <= *mid) ++mid;
        w.erase(w.lower_bound(nums[i - k]));         // erase ONE copy, not all
    }
    return ans;
}
```

Note `w.erase(w.lower_bound(x))` erases one copy; `w.erase(x)` would erase **every** copy of `x`, a classic bug. This version also passes the random tests.
- **Rolling p-th percentile**: same two-heap structure, but keep $|\text{lo}| = \lceil pk \rceil$ instead of half.
- **Sliding window maximum**: a heap works, but a monotonic deque gives $O(n)$ (see the monotonic stack digest).
- **Top-k largest of a stream**: a min-heap of size k, $O(n \log k)$.